

RAPPORT :

Pour cet exercice, j'ai écrit plusieurs petits exemples afin de comprendre comment fonctionne le message dispatch en Pharo.

Exemple 1 - Méthodes différentes selon les classes :

```
Object subclass: #Animal
  instanceVariableNames: "
  classVariableNames: "
  package: 'DispatchDemo'.
```

```
Object subclass: #Dog
  instanceVariableNames: "
  classVariableNames: "
  package: 'DispatchDemo'.
```

```
Animal >> speak
  ^ 'Un animal fait un bruit'
```

```
Dog >> speak
  ^ 'Le chien aboie'
```

```
a := Animal new.
d := Dog new.
Transcript show: a speak; cr.
Transcript show: d speak; cr.
```

Dans ce code, j'ai créé deux classes : Animal et Dog. Chaque classe a sa propre méthode speak. Quand j'exécute le code, le Transcript affiche d'abord Un animal fait un bruit puis Le chien aboie.

Exemple 2 Variable trompeuse :

```
animal := Dog new.
Transcript show: animal speak; cr.
```

Ici, j'ai nommé la variable animal mais elle contient un objet Dog. Le résultat affiché est Le chien aboie. Cela prouve que le dispatch ne se fait pas sur le nom de la variable, mais bien sur la classe réelle de l'objet.

Exemple 3 - Classe sans méthode avec héritage :

```
Object subclass: #Cat
  instanceVariableNames: "
  classVariableNames: "
  package: 'DispatchDemo'.
```

```
[ Transcript show: (Cat new speak); cr ]
on: MessageNotUnderstood
do: [ :ex | Transcript show: 'MNU: ', ex messageText; cr ].
```

J'ai créé une classe Cat sans définir speak. Quand j'appelle Cat new speak, j'ai une erreur MessageNotUnderstood. Cela veut dire que si une méthode n'existe pas dans la classe (ou ses parents), Pharo lève une erreur. Pour corriger cela, j'ai ajouté une méthode par défaut dans Animal :

```
Animal >> speak  
^ 'Un animal fait un bruit (défaut)
```

Ensuite, Cat new speak affiche Un animal fait un bruit (défaut). Donc Cat hérite de son parent Animal.

Exemple 4 - Utilisation de super ;

```
Dog >> speakLoud  
^ (super speak , ' + WOUF!)
```

Transcript show: (Dog new speakLoud); cr.

Ici, j'ai défini une nouvelle méthode dans Dog qui appelle aussi la méthode de Animal grâce à super. Le résultat est Un animal fait un bruit (défaut) + WOUF!. Cela montre comment on peut combiner le comportement de la classe parente et celui de la sous-classe.

Débrief :

Au départ, je pensais que le type supposé d'une variable allait influencer quelle méthode était exécutée, mais j'ai vu que ce n'était pas le cas. En Pharo, le dispatch est dynamique : c'est toujours la classe réelle de l'objet au moment de l'exécution qui détermine la méthode appelée. La principale différence entre ce que j'attendais et la réalité est venue du polymorphisme. Même si une variable est déclarée comme animal, si elle contient un Dog, c'est la méthode de Dog qui est exécutée. L'erreur MessageNotUnderstood avec Cat m'a aussi permis de comprendre que Pharo recherche une méthode d'abord dans la classe, puis dans ses parents, et lève une erreur si rien n'est trouvé. J'ai corrigé mes fausses idées en testant dans le Playground et en consultant les outils Implementors et Senders de l'IDE, ainsi que le livre Pharo by Example. En résumé, j'ai compris que le message dispatch en Pharo repose entièrement sur le type concret de l'objet et que l'héritage et super permettent d'organiser le code entre classes parentes et enfants.